

0.87.2. Agrandir une lettre : texteGrand — dix fois

Le plus grand : `texteGrand(5, 4, "A", 10)`. Un A de 80×80 pixels, 10×10 cases, la moitié de l'écran.

1. Ce qu'il faut comprendre

La taille la plus grande, 10 : chaque pixel devient un carré de 10×10 pixels. Le A fait **80×80 pixels**, soit **10×10 cases** : la moitié de la largeur de l'écran (20 cases).

La moitié de l'écran : l'écran fait 20×18 cases ; à la taille 10, une lettre en prend 10×10 . La plus grande taille, 20, remplit l'écran entier : c'est le 0.87.4.

Ça ne coûte presque rien de plus : aux grandes tailles, beaucoup de tuiles sont **toutes pleines** ou **toutes vides**. Les tuiles identiques ne sont fabriquées qu'une fois.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction `texteGrand\(\)` — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1700 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          55 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    489 o ← main() — ta fonction
main()             18 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AdresseCase        10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
Tuiles            944 o ← CopierTuiles — la police entière, et 15 tuiles dessinées
GRAND_TABLE_0_0_0  10 o ← poser() — une table gravée
GRAND_TABLE_0_1_1  10 o ← poser() — une table gravée
GRAND_TABLE_0_2_2  10 o ← poser() — une table gravée
GRAND_TABLE_0_3_3  10 o ← poser() — une table gravée
GRAND_TABLE_0_4_4  10 o ← poser() — une table gravée
GRAND_TABLE_0_5_5  10 o ← poser() — une table gravée
GRAND_TABLE_0_6_6  10 o ← poser() — une table gravée
GRAND_TABLE_0_7_7  10 o ← poser() — une table gravée
GRAND_TABLE_0_8_8  10 o ← poser() — une table gravée
GRAND_TABLE_0_9_9  10 o ← poser() — une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
```

EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()          × 10   308 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (10 o),  
GRAND_TABLE_0_1_1 (10 o), GRAND_TABLE_0_2_2 (10 o), GRAND_TABLE_0_3_3 (10 o), GRAND_TABLE_0_4_4 (10 o),  
GRAND_TABLE_0_5_5 (10 o), GRAND_TABLE_0_6_6 (10 o), GRAND_TABLE_0_7_7 (10 o), GRAND_TABLE_0_8_8 (10 o),  
GRAND_TABLE_0_9_9 (10 o)
```

```
image()          × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
texte_grand_0() × 1     3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

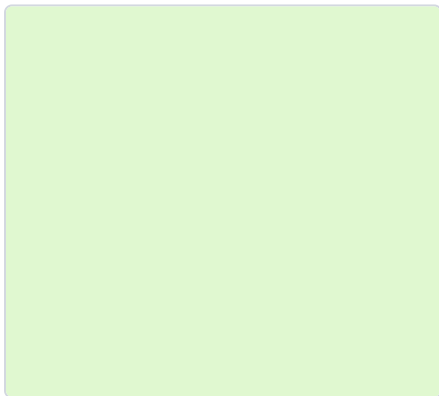
```
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    // Le A dix fois plus grand : 10 × 10 cases, de (5, 4) à (14, 13).  
    texteGrand(5, 4, "A", 10);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A géant au milieu de l'écran, dix fois plus grand qu'une lettre normale.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A géant remplit une bonne part de ses 10 × 10 cases (46)

✓ il ne déborde pas

Ensuite : **0.87.3. Agrandir une lettre : `texteGrand` — un mot** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche
Game Boy en C++